FATEC POMPEIA

Introdução à Computação em Nuvem · ISW055 · 161_SIST. INTELIGENTES_N

P2

Relatório consolidado

arquitetura e atividades do 2º bimestre

Avaliação individual · 2026.2

Ígor Rodrigues Rueda

Produto: **Atena SGE - Sistema de Gerenciamento Empresarial**
https://github.com/Igor-Rueda/Atena-SGE

Prof. Allan Lincoln Rodrigues Siriani
Pompeia, 13/11/2026

Sumário

1 Introdução 2
2 Metodologia 2
3 Arquitetura do SaaS 3
3.1 Visão geral e componentes 4
3.2 Onde roda 5
3.3 Fluxo de uma requisição 6
3.4 Fluxo de assinatura 8
3.5 Modelo de dados do domínio 9
3.6 Segurança 9
3.7 Observabilidade, CI/CD e documentação da API 12
3.8 Camadas IaaS, PaaS e SaaS do projeto 12
4 Quadro de entregas 13
5 Atividades realizadas 13
5.1 Atividade 7 — Plano Premium — cobrança de verdade com Stripe 13
5.2 Atividade 8 — SaaS, PaaS, IaaS — e a documentação que o projeto nunca teve 16
5.3 Atividade 9 — Fork e plano de negócio 18
5.4 Atividade 10 — Seu negócio no ar — o domínio no lugar dos filmes 20
5.5 Atividade 11 — SaaS completo — planos, limites e assinatura com Stripe 25
6 Considerações finais 28
7 Declaração de autoria 28

1 Introdução

Este relatório documenta o desenvolvimento final do projeto da disciplina, cuja proposta é aplicar conceitos de arquitetura de software e infraestrutura em sistemas em nuvem, visando a construção de uma solução completa e comercial. O trabalho teve início com o desenvolvimento de uma aplicação fundamental — um catálogo de filmes focado no consumo de APIs —, mas evoluiu substancialmente ao longo do período para uma arquitetura baseada em microsserviços, transformando-se no Atenas SGE, um Software as a Service (SaaS) robusto.

O Atenas SGE foi projetado para solucionar um gargalo operacional crítico em pequenas indústrias: a gestão fragmentada de demandas e a sobrecarga da equipe produtiva. O público-alvo central são os encarregados de produção que atualmente dependem de planilhas isoladas e grupos de WhatsApp para distribuir tarefas entre os processos de planejamento, almoxarifado, fabricação e montagem. O software centraliza os projetos e demandas, entregando a cada operador apenas a informação necessária para o seu trabalho. Seu principal diferencial competitivo é a eliminação da adivinhação gerencial: o sistema utiliza o histórico da própria empresa para calcular o número de funcionários exigidos para a meta do mês, evitando ociosidade e horas extras.

Nas seções a seguir, este documento detalha a concepção e a implementação desta plataforma. O relatório está organizado de forma a apresentar, inicialmente, a modelagem de negócios, os papéis de usuário e a reestruturação do banco de dados. Em seguida, descreve a arquitetura técnica, abordando a segregação de permissões, o fluxo de assinaturas integrado ao Stripe e a construção da interface gráfica. Por fim, o documento aprofunda a implementação do serviço isolado de análise de dados — uma regressão linear múltipla em Python para previsão de equipes — e conclui com o roteiro de validação e o planejamento das próximas iterações.

2 Metodologia

As atividades 7 e 8 foram feitas no repositório público do catálogo de filmes, o mesmo do 1º bimestre. Na atividade 9, esse repositório foi copiado com todo o histórico (fork do github) para um repositório novo, com o nome do produto, onde o SaaS foi desenvolvido nas atividades 10 e 11, na mesma stack do Portainer e no mesmo domínio. A data e a hora de cada entrega foram extraídas do histórico de commits (`git log`), conferidas na página do commit no GitHub e, da atividade 9 em diante, na issue de entrega que menciona o professor. Os prints mostram o README com a menção ao professor, o histórico de commits e o sistema em execução.

O desenvolvimento e a implantação do sistema foram realizados no ambiente de infraestrutura definido pela disciplina. A aplicação final (Atenas SGE) foi conteinerizada via Docker Compose e hospedada na stack do Portainer, mantendo a publicação no mesmo domínio utilizado nas entregas do bimestre anterior. Para o armazenamento persistente, o sistema conectou-se ao banco de dados relacional disponibilizado remotamente pelo professor, que serviu como base centralizada para todas as tabelas de negócio e registros de acesso.

O fluxo de trabalho e a validação das etapas basearam-se no controle de versão e no gerenciamento de projeto do repositório. O escopo, os requisitos e os critérios de aceite de cada ciclo de desenvolvimento foram guiados estritamente pelas exigências descritas no arquivo README.md. As entregas correspondentes a essas exigências foram formalizadas e rastreadas por meio da abertura e conclusão de issues no GitHub.

A comprovação dos prazos foi feita mediante a coleta da data e da hora exatas do histórico de commits e da movimentação das issues onde o professor foi mencionado para avaliação. Os prints anexados a este documento evidenciam a conformidade do processo, demonstrando os requisitos no README, a trilha de execução no GitHub e o software operando plenamente no ambiente do Portainer.

3 Arquitetura do SaaS

A arquitetura em produção do Atenas SGE é baseada num ecossistema de microsserviços conteinerizados, orquestrados via Docker Compose e implantados de forma automatizada. O sistema não utiliza um monólito único, distribuindo as responsabilidades por serviços modulares fechados numa rede interna, onde apenas a aplicação principal está exposta ao cliente.

Abaixo encontra-se a descrição detalha de cada componente da Stack:

- **Atenas App (app)**: O único serviço com porta pública (3000). Funciona simultaneamente como front--end e API Gateway, concentrando a interface do utilizador, o CRUD gratuito de projetos/demandas e o encaminhamento das requisições internas (como webhooks e autenticação) para os restantes serviços.
- **Auth Service (auth-service)**: Microsserviço interno responsável por gerir o registo de utilizadores, login (com emissão de JWT) e recuperação de palavras-passe.
- **Pagamentos (pagamentos)**: Serviço interno que processa a criação das sessões de checkout e a validação do webhook do Stripe. O tráfego do Stripe chega primeiro ao app e é repassado internamente para validação das assinaturas.
- **Profile Service (profile-service)**: Gere a biografia do utilizador e as fotos de perfil, atuando como ponte direta com o armazenamento de objetos.
- **Regressão (regressao)**: Microsserviço em Python focado em Data Science e Business Intelligence (BI). Recebe ficheiros CSV do utilizador, treina os modelos de regressão linear múltipla em tempo real e devolve a previsão de equipa necessária.
- **Log Service (log-service)**: Consome os eventos de negócio e de auditoria de forma assíncrona, realizando os registros das ações dos usuários.
- **MariaDB (Banco compartilhado)**: Base de dados relacional partilhada onde residem todas as tabelas (utilizadores, projetos, itens, clientes do Stripe e resultados dos modelos de regressão treinados).
- **Redis**: Message broker em memória utilizado primariamente para encaminhar os eventos de auditoria (Redis Streams) até ao Log Service.
- **Garage S3**: Solução de object storage compatível com a API do S3 da AWS, utilizada pelo profile-service para guardar os anexos e ficheiros multimídia de forma estática e independente.

3.1 Visão geral e componentes

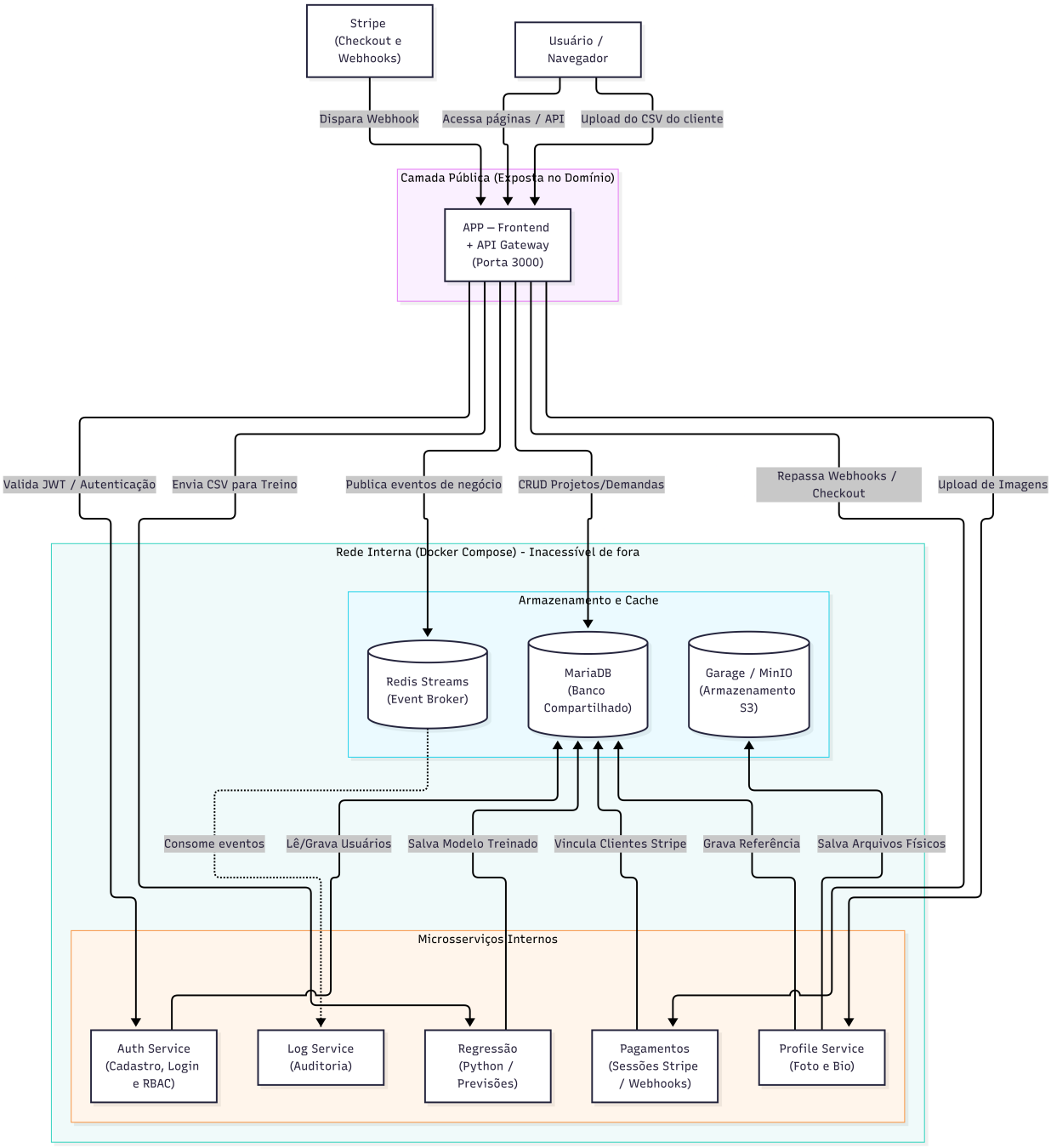


Figura 1: Diagrama de componentes e microsserviços do SaaS

Componente	Responsabilidade	Tecnologia / imagem	Público?
app	telas, API do negócio, landing page	igorrueda/ atenas-app	sim (https://igor-rueda--isw055.lapps.studio)

Componente	Responsabilidade	Tecnologia / imagem	Público?
auth-service	login, cadastro, papéis (RBAC)	igorrueda/ atenas-auth- -service	Não
log-service	eventos de negócio e auditoria	Imagem: igorrueda/ atenas-log- -service	Não
Redis	armazenamento dos logs	redis	Não (na aplicação, a imagem oficial é publica)
MariaDB	dados do domínio, usuários, planos	Tecnologia: Banco de dados relacional (SQL),Imagem: mariadb	Externo - banco compartilhado, restrito aos alunos e ao professor
Garage	arquivos do domínio	Tecnologia: Object Storage ,Imagem: dxflrs/garage	Não (na aplicação, a imagem oficial é publica)
Stripe (externo)	checkout, assinaturas, portal, webhooks	serviço SaaS	webhook → app
Profile Service	Upload de foto de perfil	igorrueda/ atenas-profile- -service	Não
Regressão	Treinar modelo de regressão linear múltipla	igorrueda/ atenas-regressao	Não

3.2 Onde roda

A aplicação roda na mesma stack utilizada no começo da disciplina, por isso o nome desta é **tom-hanks- -stack**, nome este que fazia juz a aplicação hospedada anteriormente (API-Tom-Hanks). A stack puxa o código diretamente das imagens do docker-hub, declaradas no docker-compose dentro da stack

Abaixo, os nomes dos containeres e o uso de memória (não tenho acesso aos limites de memória, dados do dia 13/11 às 20:30):

- tom-hanks-stack-app-1 ≃ 31.7 MB
- tom-hanks-stack-auth-service-1 ≃ 27.9 MB
- tom-hanks-stack-garage-1 ≃ 10 MB
- tom-hanks-stack-log-service-1 ≃ 18.1 MB

- tom-hanks-stack-pagamentos-1 ≃ 28.5 MB
- tom-hanks-stack-profile-service-1 ≃ 21.1 MB
- tom-hanks-stack-regressao-1 ≃ 82.3 MB

Variáveis de ambiente:

- PORT_APP=
- PORT_AUTH=
- PORT_PROFILE=
- PORT_PAGAMENTOS=
- PORT_LOG=
- PORT_REGRESSAO=
- AUTH_SERVICE_URL=
- PROFILE_SERVICE_URL=
- PAGAMENTOS_SERVICE_URL=
- LOG_SERVICE_URL=
- REGRESSAO_SERVICE_URL=
- APP_URL=
- JWT_SECRET:
- INTERNAL_API_KEY:
- JWT_SECRET=
- INTERNAL_API_KEY=
- DB_HOST=
- DB_USER=
- DB_PASSWORD=
- DB_NAME=
- REDIS_HOST=
- REDIS_PORT=
- SMTP_HOST=
- SMTP_PORT=
- SMTP_USER=
- SMTP_PASS=
- STRIPE_SECRET_KEY=
- STRIPE_WEBHOOK_SECRET=
- STRIPE_PRICE_EXPERT=
- GARAGE_ENDPOINT=
- GARAGE_REGION=
- GARAGE_BUCKET=
- GARAGE_ACCESS_KEY_ID=
- GARAGE_SECRET_ACCESS_KEY=
- PROFILE_IMAGE_MAX_SIZE_MB=

A aplicação está disponível na seguinte URL pública: **https://igor-rueda-isw055.lapps.studio**

3.3 Fluxo de uma requisição

Abaixo um lista numerada que descreve o fluxo da aplicação:

1. Navegador (Cliente): O utilizador, autenticado com a categoria de Planejador, preenche os dados (RG, material, descrição e quantidade) na página do projeto e clica em cadastrar. O navegador faz uma

requisição HTTP POST para o domínio público (na porta 3000), enviando os dados e o token JWT da sessão.

2. Atenas App (API Gateway): A requisição chega ao serviço app. Ao identificar que se trata de uma rota protegida do negócio, o app interrompe temporariamente o fluxo para validar quem está a fazer o pedido.
3. Auth Service (Autenticação e RBAC): O app envia o token JWT, através da rede interna, para o auth--service. Este serviço valida a assinatura criptográfica do token e devolve as claims do utilizador — confirmando a sua identidade, o plano da conta e, criticamente, a sua categoria (Planejador). Como a categoria permite criar demandas, a operação é autorizada (se fosse um Montador, por exemplo, o fluxo seria interrompido com um erro 403 Forbidden).
4. MariaDB (Persistência): De volta ao app, a regra de negócio é executada. O serviço faz um INSERT na tabela demandas do MariaDB (banco de dados partilhado), registando a nova tarefa, vinculando-a ao projeto e definindo o seu estado inicial (que entrará no processo de Almoxarifado).
5. Redis Streams (Emissão de Evento): Após a confirmação de gravação no banco, o app dispara uma mensagem assíncrona para o broker de eventos Redis, publicando o evento demanda_criada com os dados do utilizador e da tarefa.
6. Log Service (Auditoria): O log-service, que opera de forma isolada na rede interna consumindo o Redis Streams, recebe a mensagem e grava o evento de auditoria no seu próprio registo, garantindo o histórico de ações sem atrasar a resposta ao utilizador.
7. Resposta (Navegador): Em paralelo à auditoria, o app finaliza a requisição e devolve um HTTP Status 201 Created (juntamente com o JSON da demanda registada) de volta ao navegador, que atualiza a interface e exibe a nova demanda na fila do projeto.

3.4 Fluxo de assinatura

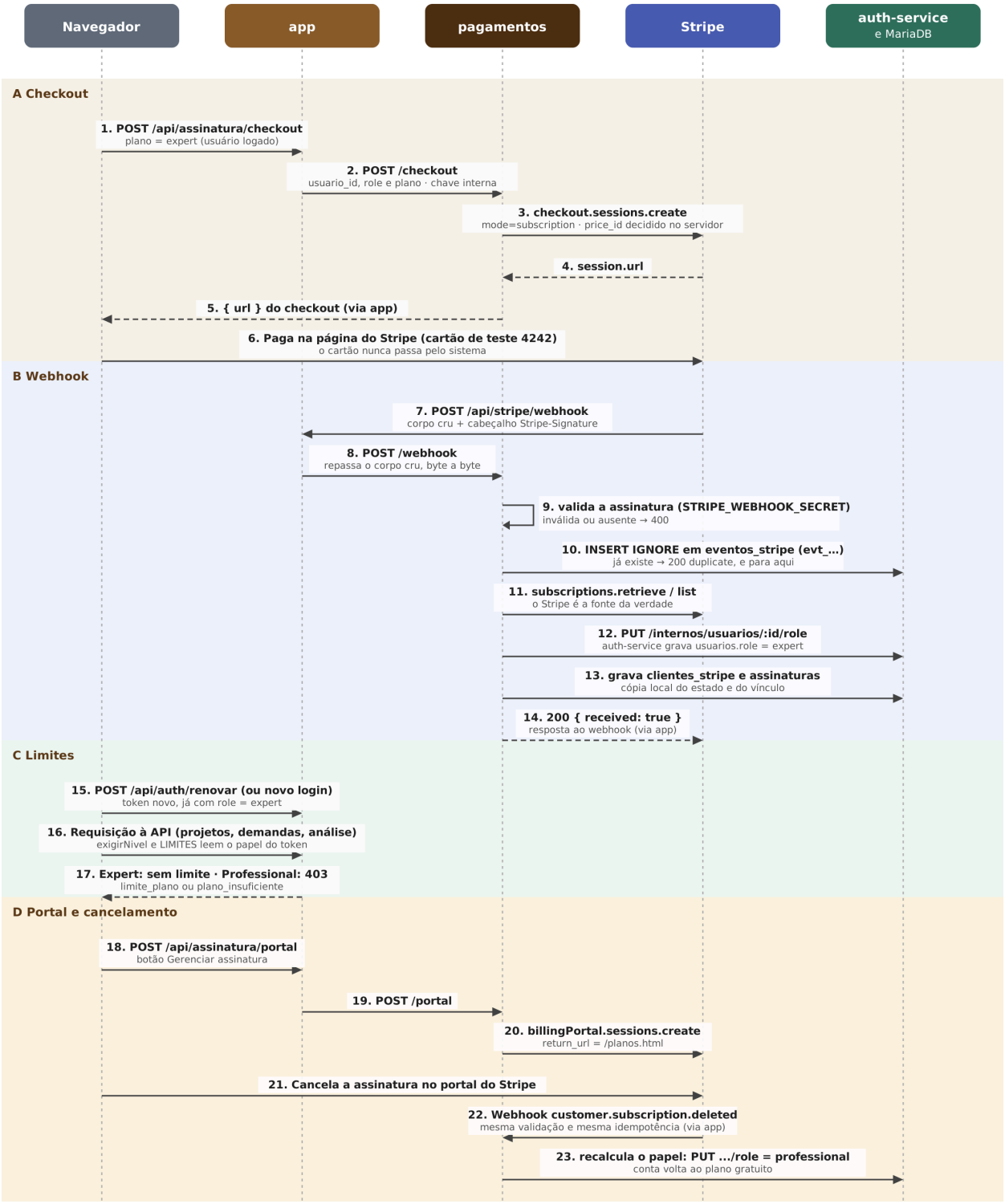


Figura 2: Fluxo de assinatura: checkout ⟶ webhook ⟶ plano no banco

3.5 Modelo de dados do domínio

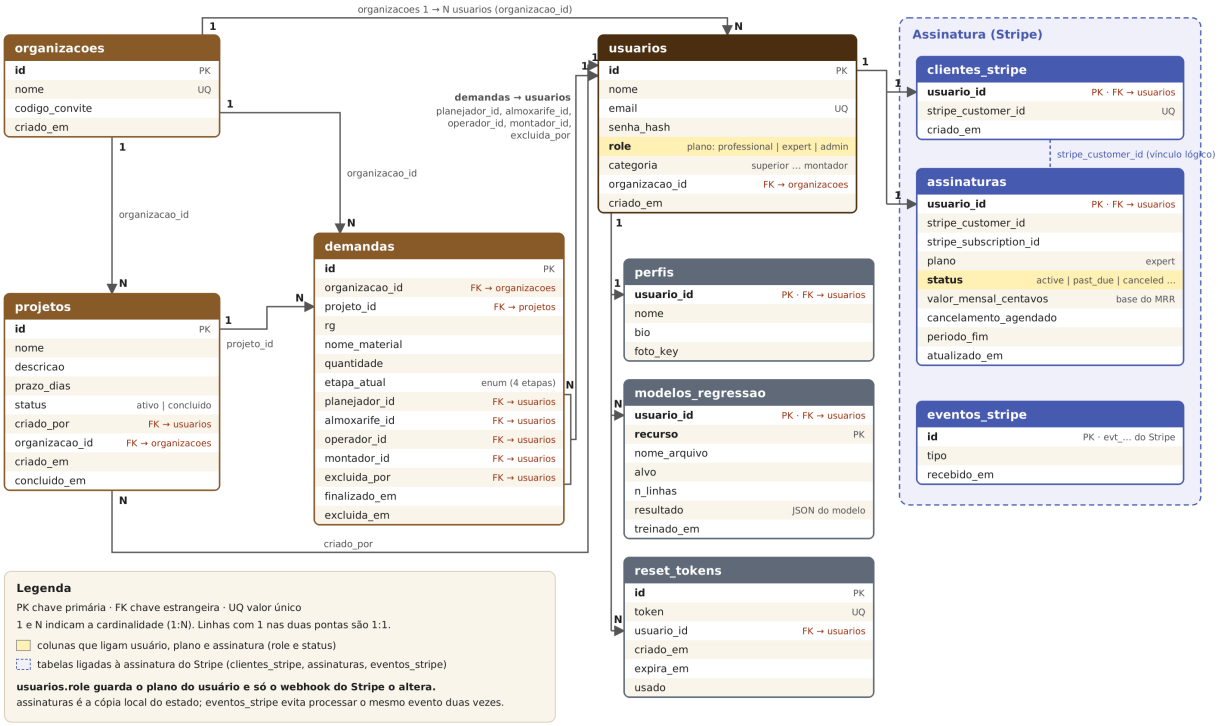


Figura 3: Modelo de dados do domínio

3.6 Segurança

A segurança do Atenas SGE se apoia em três regras: a permissão é conferida na API (a tela só esconde o que a pessoa não pode fazer), cada empresa só enxerga e altera os próprios dados, e nenhum segredo vai para o repositório.

3.6.1 Papéis e permissões

Há dois eixos de permissão, independentes. O **papel do plano** (`professional`, `expert` e `admin`) é cumulativo e decide os limites e os recursos de análise. A **categoria** do usuário (Superior, Planejador, Almoxarife, Operador e Montador) decide quem faz o quê no fluxo de projetos e demandas. Ação proibida responde `403`, mesmo por chamada direta à API, e fica registrada no log como `acesso_negado`.

Ação	admin	expert	professional
Cadastrar, editar e excluir projetos e demandas (conforme a categoria)	sim	sim	sim, com limites
Passar de 3 projetos ativos ou de 50 demandas abertas	sim	sim	não
Ver projetos, demandas e perfis da própria empresa	sim	sim	sim
Ver dados de outra empresa	não	não	não
Editar o perfil de outra pessoa	não	não	não
Treinar e consultar a previsão de equipe	sim	sim	não
Ver os indicadores da produção	sim	sim	não
Assinar o plano Expert	não	não (já tem)	sim
Gerenciar a assinatura no portal do cliente	não	sim	se já assinou
Consultar o log de auditoria	sim	não	não
Ver o painel admin (usuários por plano, assinaturas e MRR)	sim	não	não

Tabela 1: Permissões por papel do plano.

O `admin` é o administrador do produto, usado para testar rotas. Ele não é vendável, não aparece na página de planos e só nasce por comando no banco. Mesmo ele não escapa do isolamento: os indicadores e os dados de negócio continuam filtrados pela sua empresa, e o painel admin mostra apenas totais do produto, de propósito.

Ação	Superior	Planejador	Almoxarife	Operador	Montador
Cadastrar projetos	sim	não	não	não	não
Editar, excluir e concluir projetos (só os que criou)	sim	não	não	não	não
Ver os projetos e as demandas da empresa	sim	sim	sim	sim	sim
Ver e renovar o código da empresa	sim	não	não	não	não
Cadastrar demandas	não	sim	não	não	não
Editar e excluir demandas (só as que criou, não finalizadas)	não	sim	não	não	não
Avançar a demanda	não	não	Almoxarifado	Operação	Montagem
Usar a fila “Minhas demandas”	não	não	sim	sim	sim

Tabela 2: Permissões por categoria de usuário, dentro da empresa.

3.6.2 Isolamento entre empresas

A unidade de isolamento é a empresa, e não o usuário isolado: a equipe de uma mesma empresa compartilha projetos e demandas, e nenhuma empresa enxerga os dados de outra. Dentro da empresa, os dados pessoais continuam individuais: o modelo de previsão pertence a quem o treinou, cada pessoa edita só o próprio perfil, e a aba Excluídas mostra ao Planejador apenas as demandas que ele excluiu. O isolamento é garantido assim:

1. **A empresa vem do banco, nunca do cliente.** Depois de validar o token, a API lê `organizacao_id` e a categoria do usuário na tabela `usuarios` a cada requisição. Nem o token nem o corpo da requisição dizem a qual empresa a pessoa pertence, e uma conta sem empresa é barrada com `403`.

2. **Toda consulta e toda escrita filtram pela empresa.** Projetos, demandas, fila, indicadores e perfis usam o `organizacao_id` na própria cláusula `WHERE`, inclusive nos `UPDATE` e `DELETE`, como neste exemplo:

   ```
   SELECT * FROM projetos WHERE id = ? AND organizacao_id = ?
   ```
3. **ID de outra empresa responde `404`, não `403`.** Trocar o número na URL não devolve o registro alheio, e a resposta é igual à de um registro que não existe, para não revelar que ele existe. A tentativa fica no log (`acesso_negado`, com `motivo=outra_empresa`).
4. **Entrar numa empresa exige o código dela.** O Superior cria a empresa ao se cadastrar. Os demais informam o nome e o código de 12 caracteres, gerado de forma imprevisível, que só o Superior vê e pode renovar (o código antigo deixa de valer na hora). Nome ou código errado responde igual (`403`), o cadastro tem limite de 10 tentativas por minuto, e o `organizacao_id` enviado pelo formulário é descartado. O `auth-service` só aceita um `organizacao_id` vindo do app, com a chave interna.
5. **As permissões por categoria valem dentro da empresa.** A categoria também é lida do banco, e as regras de autoria são conferidas na API: só o Superior que criou o projeto o edita, exclui ou conclui, e só o Planejador que criou a demanda a altera. O modelo de previsão é sempre buscado pelo id do usuário do token, nunca por um id enviado na requisição.

O isolamento foi verificado num teste de integração com a API real contra o MariaDB (80 verificações). O pipeline de CI executa, a cada push, uma versão enxuta: com duas empresas, confirma que o código errado é recusado (`403`) e que a empresa B recebe `404` ao abrir, editar, concluir e excluir um projeto da A, ao listar projetos e ao pedir o perfil de um usuário da A.

3.6.3 Outros controles

- **Sessão:** o token (JWT) vai em cookie `httpOnly`, `secure` e `sameSite=strict`, e expira em 15 minutos.
- **Senhas:** guardadas só como hash (`bcrypt`). O login e a recuperação de senha têm limite de 5 tentativas a cada 15 minutos por IP, e o link de redefinição vale 30 minutos e uma única vez.
- **Serviços internos:** só o `app` tem porta pública. `auth-service`, `pagamentos`, `log-service`, `profile-service` e `regressao` ficam na rede interna e exigem a `INTERNAL_API_KEY`, comparada em tempo constante. Sem a variável, nada passa.
- **Pagamentos:** o cartão é digitado na página do Stripe e nunca passa pelo sistema. O webhook só é aceito com a assinatura do Stripe válida (seção “Fluxo de assinatura”).
- **Auditoria:** os eventos de negócio e de segurança vão para o `log-service` com quem fez, quando e a empresa, e só o `admin` consulta.

3.6.4 Segredos e configuração

Nenhum segredo fica no repositório. Em produção, eles são variáveis de ambiente da stack no Portainer. Em desenvolvimento, ficam num arquivo `.env` local, fora do versionamento, criado a partir do `env.example`. O `env.example` está no repositório e traz apenas os nomes das variáveis, sem valores reais:

Variáveis	Para que servem
`JWT_SECRET`, `INTERNAL_API_KEY`	Assinatura da sessão e comunicação entre os serviços.
`STRIPE_SECRET_KEY`, `STRIPE_WEBHOOK_SECRET`, `STRIPE_PRICE_EXPERT`	Chave do Stripe, segredo de validação do webhook e preço do plano Expert. A chave e o segredo ficam só na stack, nunca no repositório.
`DB_HOST`, `DB_USER`, `DB_PASSWORD`, `DB_NAME`	Acesso ao MariaDB.
`SMTP_HOST`, `SMTP_PORT`, `SMTP_USER`, `SMTP_PASS`	Envio do e-mail de recuperação de senha.
Credenciais do Garage	Storage S3 das fotos de perfil. O `garage.toml` real também não é versionado.

Tabela 3: Variáveis de ambiente do sistema

No pipeline de CI, o `.github/ci/.env.ci` traz apenas valores falsos e descartáveis (por exemplo, `sk_test_ci_fake`), nunca reaproveitados em produção. As credenciais do Docker Hub ficam nos segredos do GitHub Actions. Os prints de evidência não mostram `.env`, tokens, senhas nem chaves.

3.7 Observabilidade, CI/CD e documentação da API

Implementei somente o serviço de CI/CD.

Este serviço permite que à partir do push para o repositório, o código seja testado automaticamente e, por conseguinte, seja feito seu deploy para o Docker HUB. Com a imagem presente no Docker HUB, basta apenas clicar em “Update the stack” + “Re-pull image and redeploy” para atualizar a aplicação no Portainer.

O pipeline estruturado no ficheiro de configuração funciona como uma dupla barreira de qualidade: a primeira etapa cria um ambiente de testes descartável e isolado, aplicando o esquema da base de dados para validar de ponta a ponta o registo de utilizadores, a segurança dos pagamentos via Stripe e as previsões do modelo de regressão em Python; apenas se estes testes passarem sem erros é que a segunda etapa avança com a construção e envio das imagens de todos os microsserviços para o repositório de imagens, garantindo que nenhuma falha seja acidentalmente publicada.

3.8 Camadas IaaS, PaaS e SaaS do projeto

Camada	O que é no seu projeto	Quem gerencia
IaaS	VM, a rede e o disco onde a stack roda	Professor/FATEC
PaaS	Docker + Portainer, MariaDB, Redis, Garage	Aluno + Professor (Nível de acesso do Portainer)
SaaS	Atenas SGE + Serviços externos (Stripe, GitHub, SMTP Gmail)	Aluno, Provedores dos serviços

4 Quadro de entregas

Nº	Atividade	Data planejada	Data realizada	Situação
7	Plano Premium — cobrança de verdade com Stripe	09/10/2026	11/10/2026 14:48	entregue com atraso
8	SaaS, PaaS, IaaS — e a documentação que o projeto nunca teve	16/10/2026	11/10/2026 16:01	entregue
9	Fork e plano de negócio	23/10/2026	DD/MM/AAAA HH:MM	Não entregue
10	Seu negócio no ar — o domínio no lugar dos filmes	06/11/2026	10/11/2026 20:34	entregue com atraso
11	SaaS completo — planos, limites e assinatura com Stripe	13/11/2026	12/11/2026 23:37	entregue

5 Atividades realizadas

5.1 Atividade 7 — Plano Premium — cobrança de verdade com Stripe

Descrição	Plano pago em modo de teste: Checkout Session do Stripe e webhook marcando o usuário como premium.
Data planejada	09/10/2026
Data realizada	11/10/2026 14:48
Situação	entregue com atraso
Evidência	GitHub — commit + print do checkout concluído e do usuário premium
Link	https://github.com/Igor-RR/API-Tom-Hanks

O que foi feito

Foi criado o serviço isolado `pagamentos`, responsável por toda a integração com o Stripe em modo de teste. Ele não tem porta pública: só o `catalogo` conversa com ele, pela rede interna do Docker, protegido pelo segredo `INTERNAL_API_KEY`. O usuário assina os planos Fan (R$ 9,90/mês), Cinéfilo (R$ 29,90/mês) ou Stalker (R$ 99,90/mês) pelo botão “Assinar” da página de Planos, que abre uma Checkout Session em modo `subscription`. A página de pagamento é hospedada pelo próprio Stripe, então o número do cartão nunca passa pelo nosso sistema. O cliente informa apenas qual plano quer, e o `price_id` é decidido no servidor.

A principal decisão foi separar o fluxo em dois caminhos. O caminho síncrono só leva o usuário ao checkout e depois a uma página de sucesso, que apenas espera e mostra o status. O caminho assíncrono é o que vale: o Stripe chama o webhook, o `pagamentos` valida a assinatura (`Stripe-Signature`), grava o vínculo na tabela `clientes_stripe` e pede ao `auth-service` que altere o papel do usuário. A página de sucesso nunca promove ninguém, senão qualquer pessoa que digitasse a URL ganharia o plano de graça.

```
commit 14d4f1a21e73220e030ac49ad0f647b6019ef7bf
Author: Igor <igordriguesrueda15@gmail.com>
Date:   Sun Oct 11 14:48:20 2026 -0300

    Corrigindo erro no README.MD à respeito do tempo de expiração do token
```

Figura 4: Atividade 7 — último commit relativo à atividade

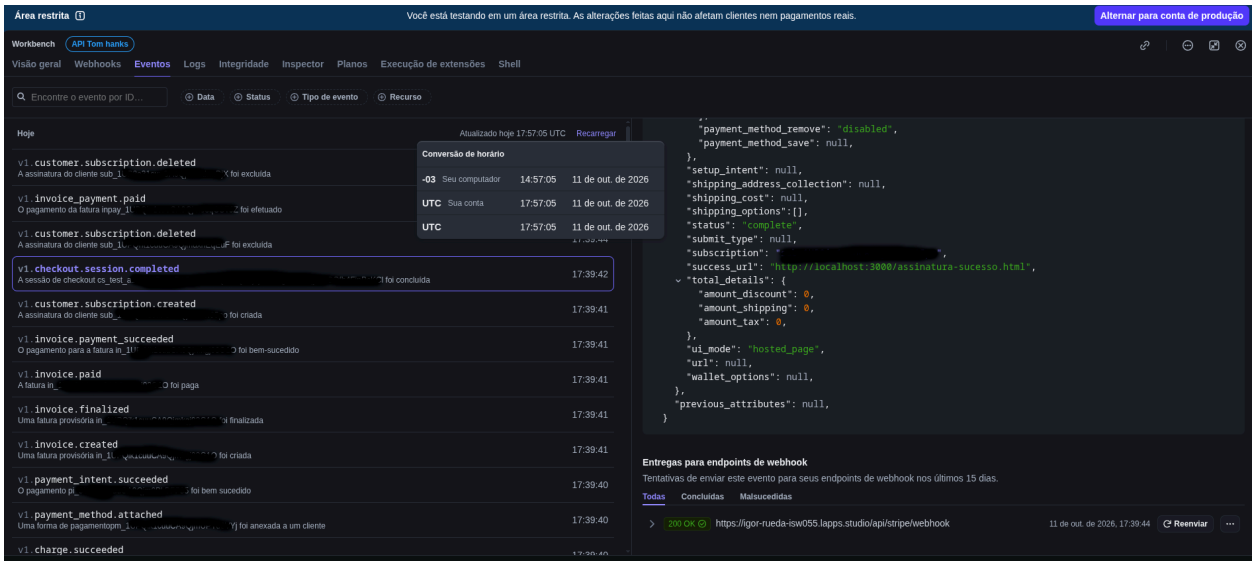


Figura 5: Atividade 7 — checkout concluído

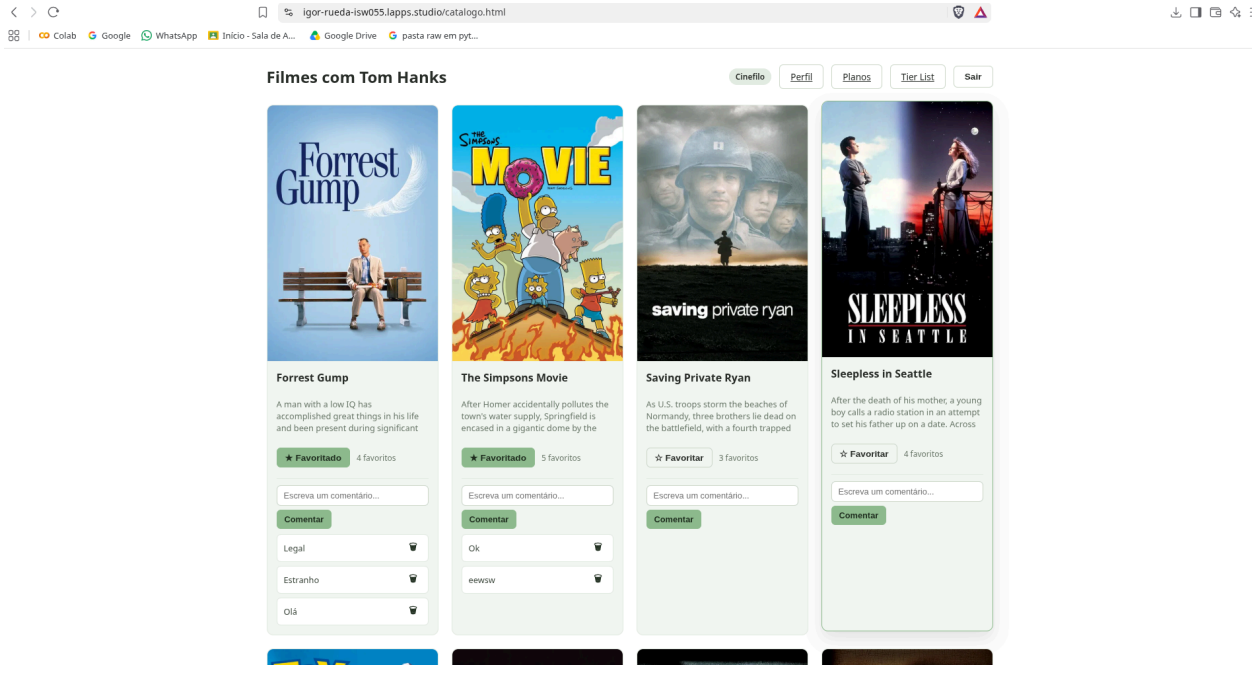


Figura 6: Atividade 7 — usuário antes do pagamento

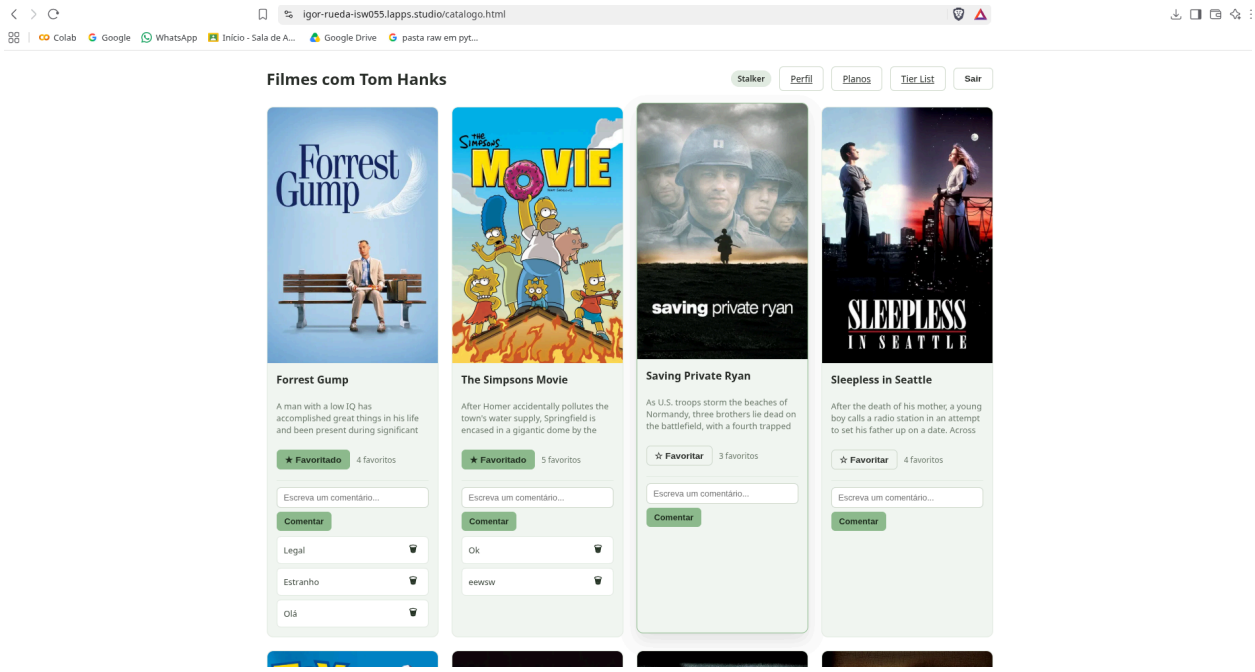


Figura 7: Atividade 7 — usuário depois do pagamento

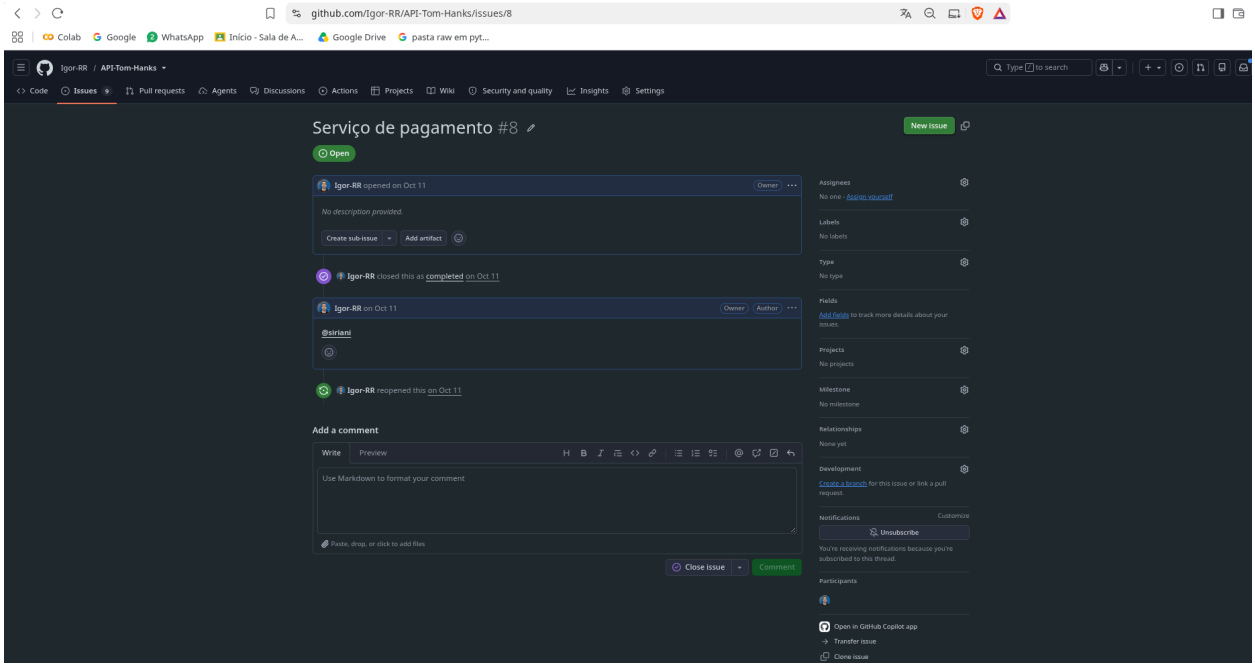


Figura 8: Atividade 7 — issue

Dificuldades e como foram resolvidas.

A maior dificuldade desta atividade foi a integração da aplicação com o webhook da stripe. Tal dificuldade foi resolvida por meio de correções no package.json e routes.js. Esta foi a primeira vez que utilizei um serviço de pagamento em uma aplicação, levando á algumas dificuldades em entender os webhooks e a necessidade de baixar a stripe CLI para testes locais.

Também houve um problema no preenchimento do README.MD (path da imagem errado), o que contribui para o atraso na entrega da atividade completa.

5.2 Atividade 8 — SaaS, PaaS, IaaS — e a documentação que o projeto nunca teve

Descrição	Camadas de nuvem do próprio projeto e documentação reversa: especificação, arquitetura, regras de negócio e modelo de dados.
Data planejada	16/10/2026
Data realizada	11/10/2026 16:01
Situação	entregue
Evidência	GitHub — commit + DOCUMENTACAO.md linkado no README
Link	https://github.com/Igor-RR/API-Tom-Hanks

O que foi feito:

Nesta atividade, foi escrita no README.md a documentação reversa do projeto, levantada lendo o código e o banco como se ele tivesse sido herdado sem documentação. O texto cobre a classificação das peças do sistema em camadas de nuvem (SaaS, PaaS e IaaS), a especificação, a arquitetura com diagrama, as regras de negócio (cada uma indicando o arquivo de onde foi extraída) e o modelo de dados.

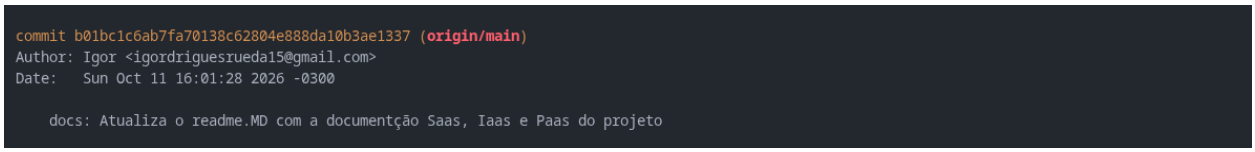
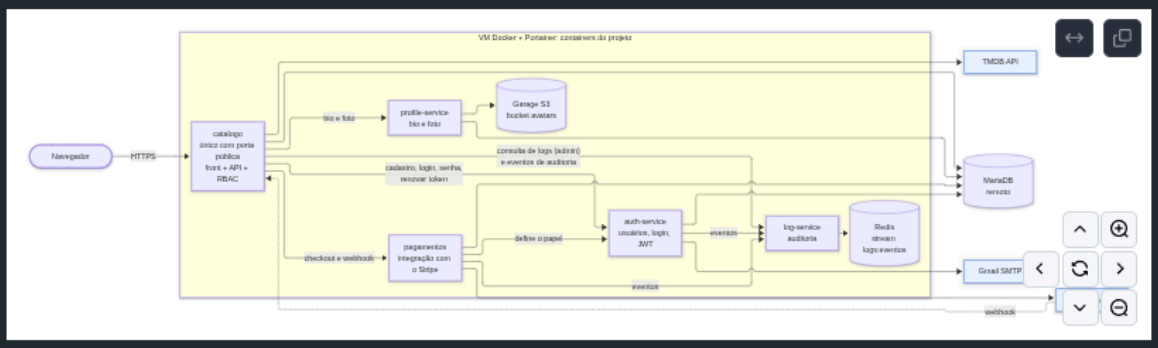


Figura 9: Atividade 8 — evidência da entrega

3. Arquitetura

O `catalogo` é o **único** serviço acessível pela internet. Todos os outros ficam na rede interna do Docker, sem porta publicada, e só são alcançados por ele.



Em azul, os serviços de terceiros (SaaS). O `catalogo` não acessa a tabela de usuários, e só o `log-service` fala com o Redis.

Serviço	Exposição	Responsabilidade	Dados que possui	Depende de
`catalogo`	Pública	Páginas, API, validação do JWT, controle de acesso por papel, proxy para os demais	`favoritos`, `comentarios`, `tier_list`	`auth-service`, `log-service`, `profile-service`, `pagamentos`, TMDB, MariaDB
`auth-service`	Interna	Cadastro, login, emissão do JWT, redefinição de senha, rotas internas de papel e token	`usuarios`, `reset_tokens`	MariaDB, SMTP, `log-service`
`log-service`	Interna	Receber e consultar eventos de auditoria	Stream `logs:eventos` no Redis	Redis
`profile-service`	Interna	Bio e foto de perfil	`perfis` e o bucket `avatars`	MariaDB, Garage
`pagamentos`	Interna	Checkout, assinaturas e webhook do Stripe	`clientes_stripe`	Stripe, `auth-service`, MariaDB, `log-service`

Figura 10: Atividade 8 — Trecho do README.MD (Diagrama da arquitetura)

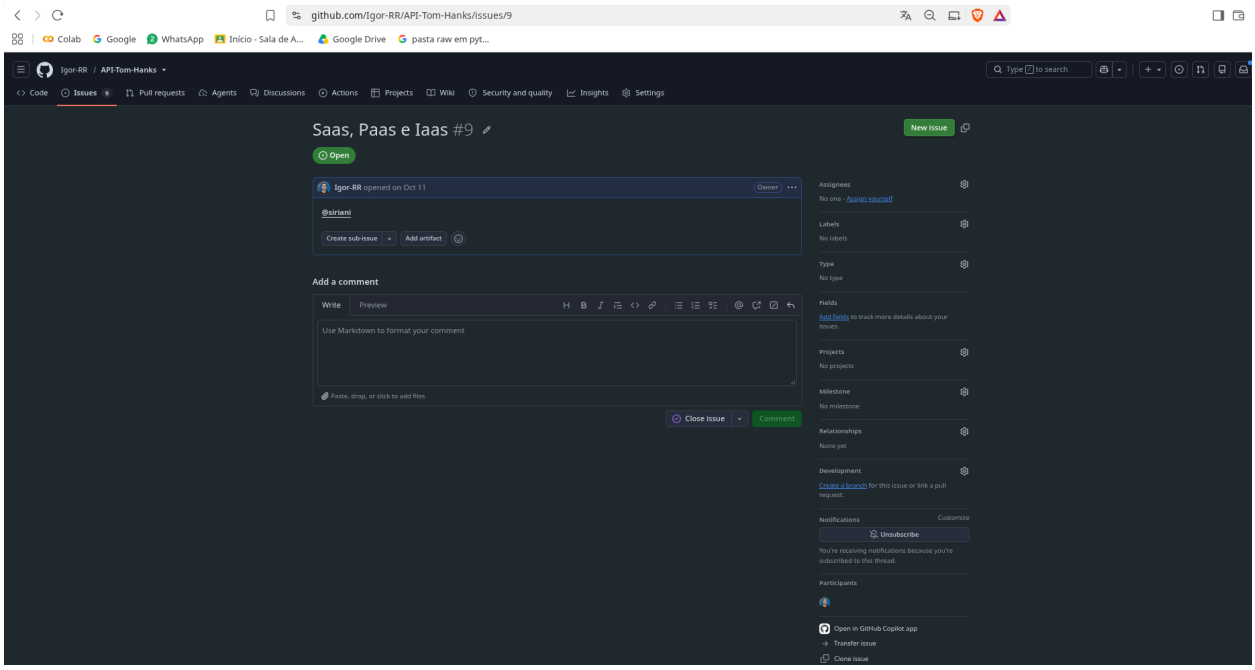


Figura 11: Atividade 8 — issue

Dificuldades e como foram resolvidas

Nesta atividade não foram encontradas grandes dificuldades para a geração da documentação, apenas pequenas dúvidas quanto a classificação (Iass, Paas e Saas) em alguns componentes da aplicação.

5.3 Atividade 9 — Fork e plano de negócio

Descrição	Repositório novo com o histórico do anterior, README do produto (dor, persona, Lean Canvas, modelo de receita, mapa de reaproveitamento) e backlog em issues.
Data planejada	23/10/2026
Data realizada	DD/MM/AAAA HH:MM
Situação	Não entregue
Evidência	GitHub — issue de entrega (@siriani) + repositório novo + README do produto + print do histórico de commits e das issues
Link	https://github.com/Igor-Rueda/Atena-SGE

O que foi feito

Inicialmente, realizei um fork do repositório original. No entanto, enfrentei problemas com a sincronização entre branches, excluindo-na indevidamente. Nesse processo, também perdi o histórico de commits original devido à uma exclusão de pastas acidental (dentre as pastas, o .git).

Apesar desse contratempo, a documentação abaixo indica o cumprimento parcial da atividade:

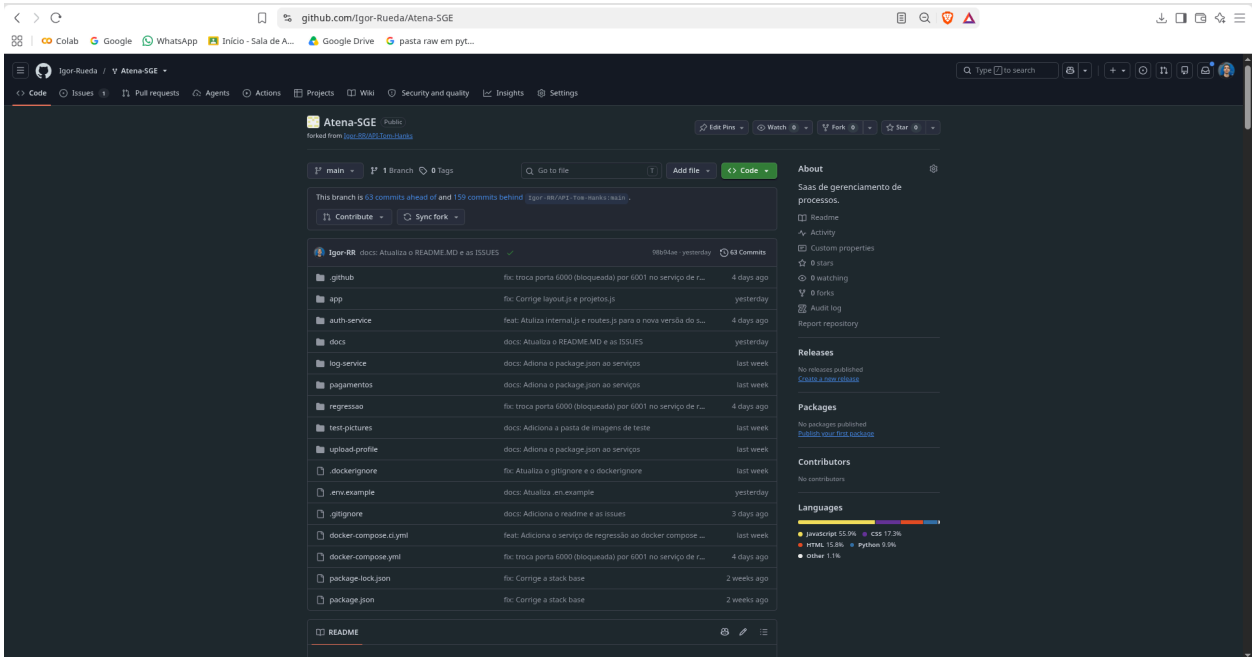


Figura 12: Atividade 9 — Repositório novo

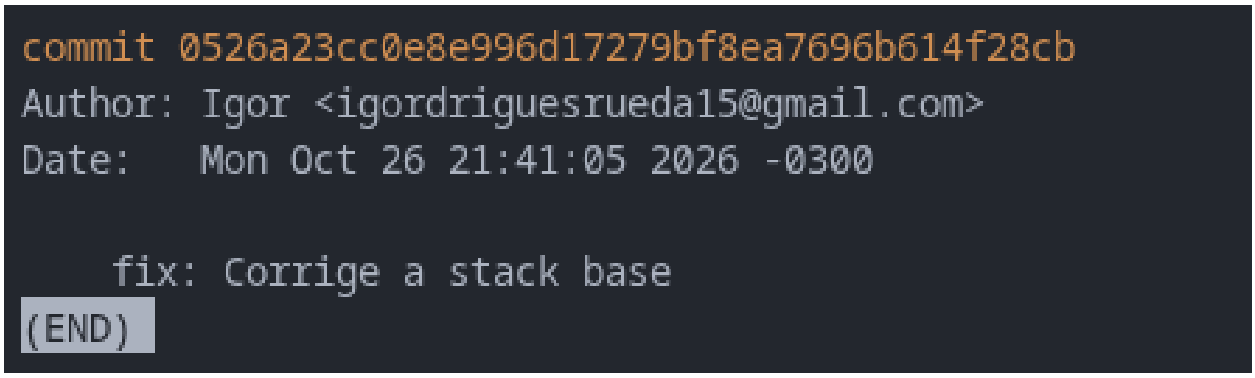


Figura 13: Atividade 9 — Commit do novo repositório (commit inicial que subiu README.MD)

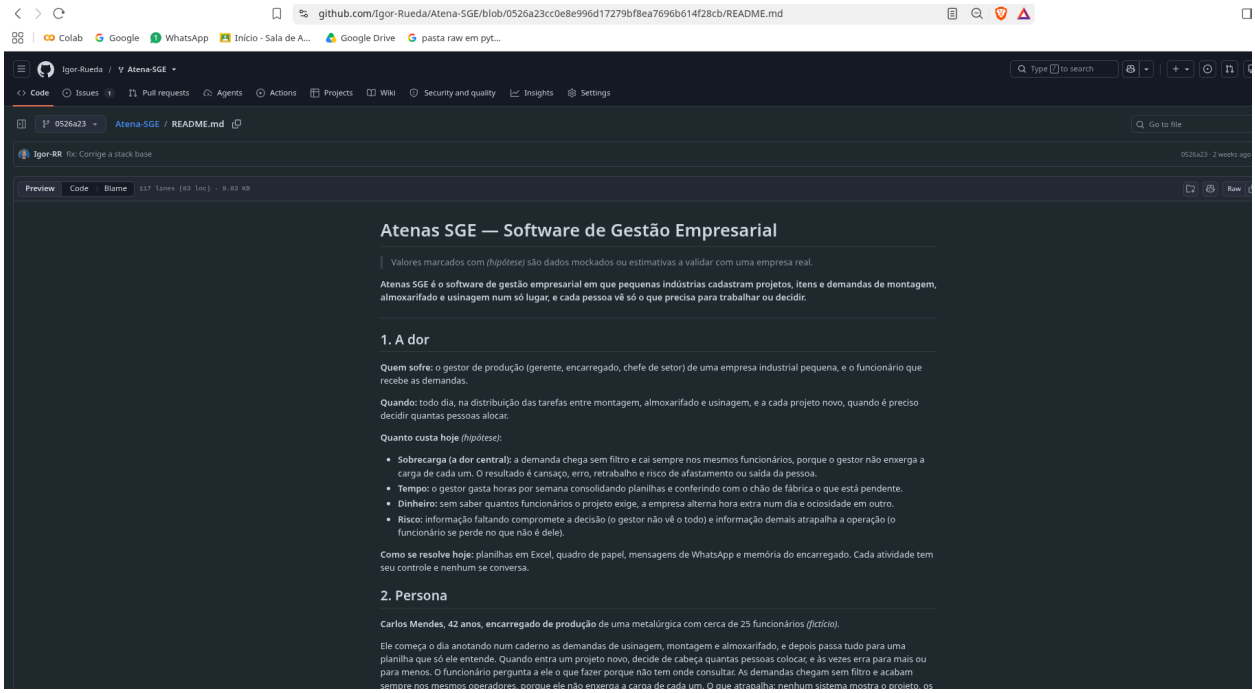


Figura 14: Atividade 9 — README.MD (Retirado do commit anexado)

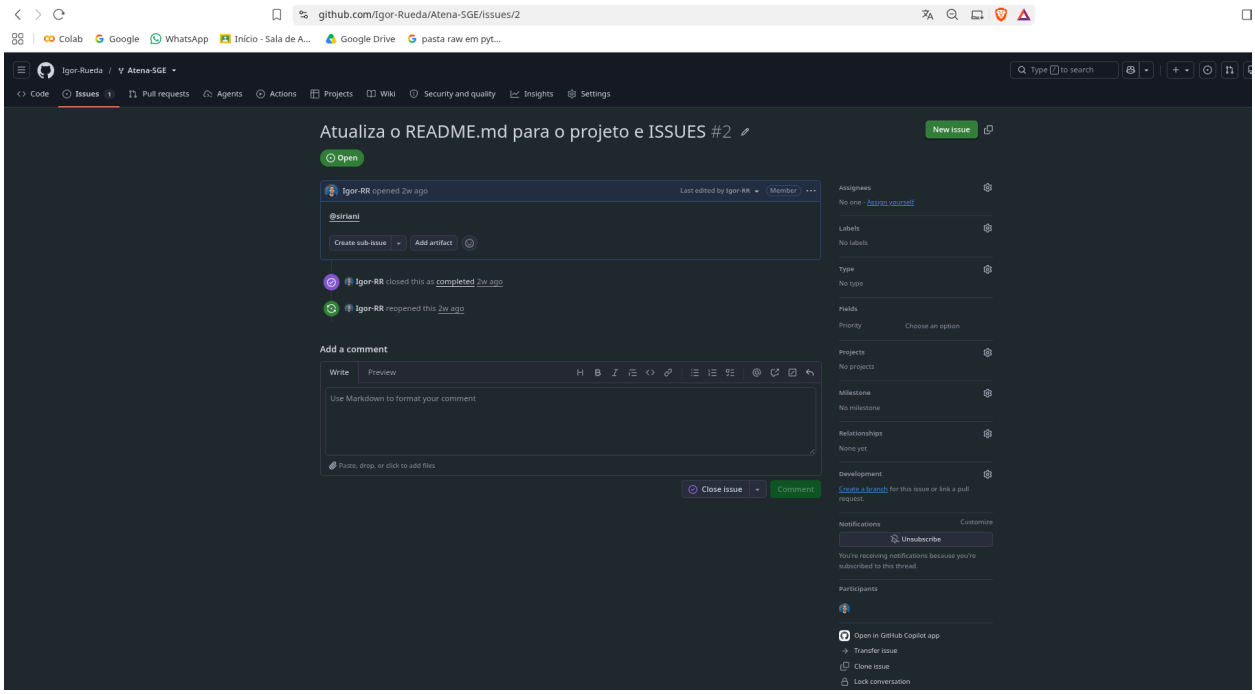


Figura 15: Atividade 9 — issues com menção ao professor

Dificuldades e como foram resolvidas

A maior dificuldade enfrentada nesta atividade foi o gerenciamento de branches e a sincronização de repositórios. Como esta foi a minha primeira experiência prática com esse tipo de fluxo de trabalho, cometi alguns equívocos operacionais em cascata: excluí acidentalmente a branch em que estava trabalhando (master) e acabei apagando o diretório .git local.

A exclusão do .git destruiu o histórico de versionamento, o que causou a incompatibilidade da origem dos commits. Para contornar a situação e não perder o projeto, precisei subir os arquivos manualmente para a branch main.

Apesar de não ter conseguido recuperar o rastreamento original devido à perda do .git, consegui corrigir os arquivos do repositório com commits posteriores.

Marco esta atividade como não concluída por não conseguir demonstra o histórico de commits no novo repositório (embora os commits do original possam ser acessados em : **https://github.com/Igor-RR/API-Tom-Hanks/commits/main/** .

5.4 Atividade 10 — Seu negócio no ar — o domínio no lugar dos filmes

Descrição	Stack apontando para o repositório novo, tabelas e API do domínio, papéis do negócio, isolamento por usuário, eventos no log, upload e landing page.
Data planejada	06/11/2026
Data realizada	10/11/2026 20:34
Situação	entregue com atraso
Evidência	GitHub — issue de entrega (@siriani) + commit + URL no ar + prints da landing, do 403 e dos eventos no log
Link	https://github.com/Igor-Rueda/Atena-SGE

O que foi feito

Nesta atividade, o domínio do sistema foi alterado do antigo catálogo de filmes para o Atenas SGE, exigindo a adaptação da arquitetura para um modelo multi-tenant. A decisão técnica central foi garantir o isolamento de dados diretamente no back-end: a API agora identifica a empresa do usuário a cada requisição e filtra as consultas, retornando erro 404 para mascarar registros de outras organizações. Além disso, implementei o controle de permissões baseado em cinco categorias operacionais. A validação ocorre na própria API, que retorna erro 403 caso um usuário tente executar ou avançar uma demanda fora de sua responsabilidade no chão de fábrica.

Na infraestrutura e nos serviços auxiliares, o sistema de auditoria em Redis foi ajustado para registrar os novos eventos de negócio, como as passagens de etapa das demandas. O armazenamento de perfis foi configurado utilizando o Garage (compatível com S3), e a landing page de entrada foi concluída. Por fim, o ambiente foi provisionado e a stack de microsserviços foi colocada no ar via Portainer, garantindo que os contêineres da API, banco de dados, logs e storage estejam operacionais e integrados.

```
commit cf61b9ba3097bd7510af70f8666a04bfdf82b99e (HEAD -> main, origin/main)
Author: Igor <igordriguesrueda15@gmail.com>
Date:   Tue Nov 10 20:34:23 2026 -0300

    docs: Atualiza o readme.md, concluindo a atividade 10
```

Figura 16: Atividade 10 — último commit relativo à atividade

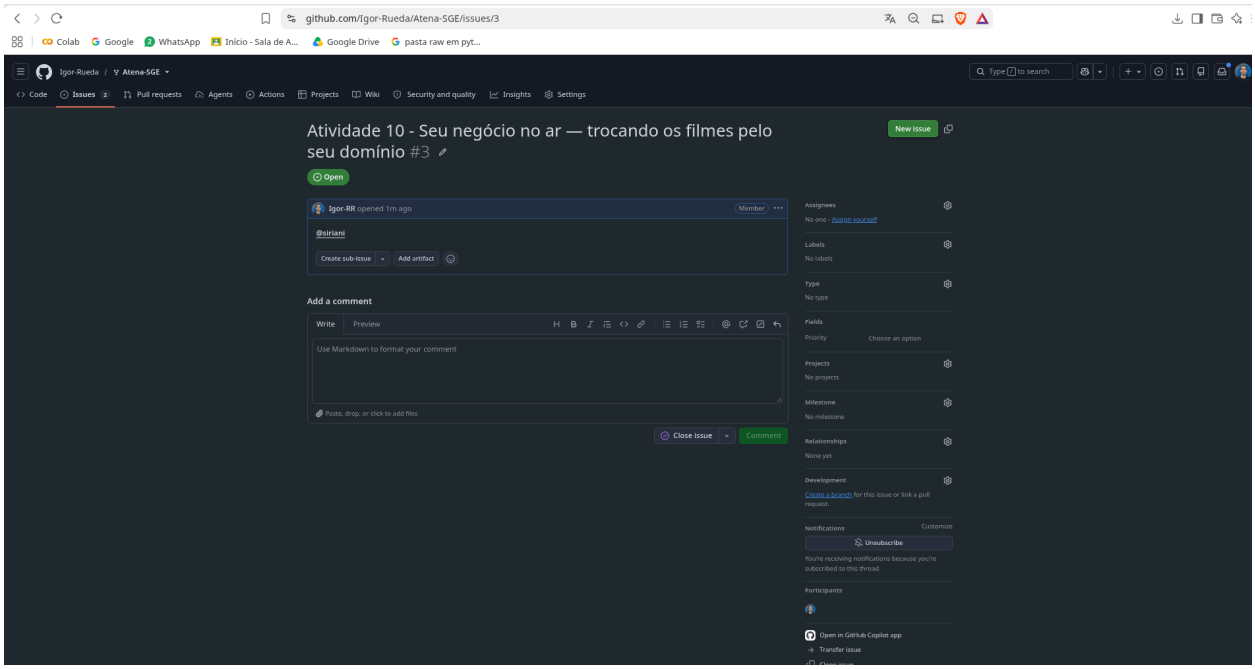


Figura 17: Atividade 10 — issue

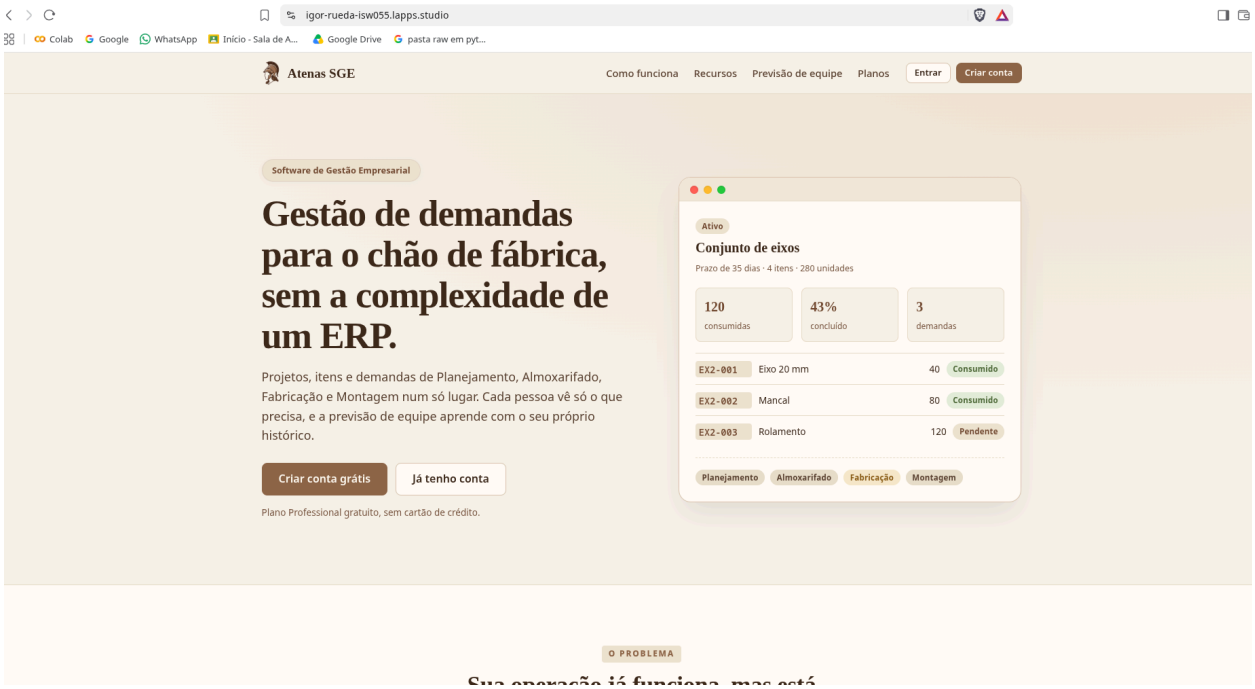


Figura 18: Atividade 10 — landing page

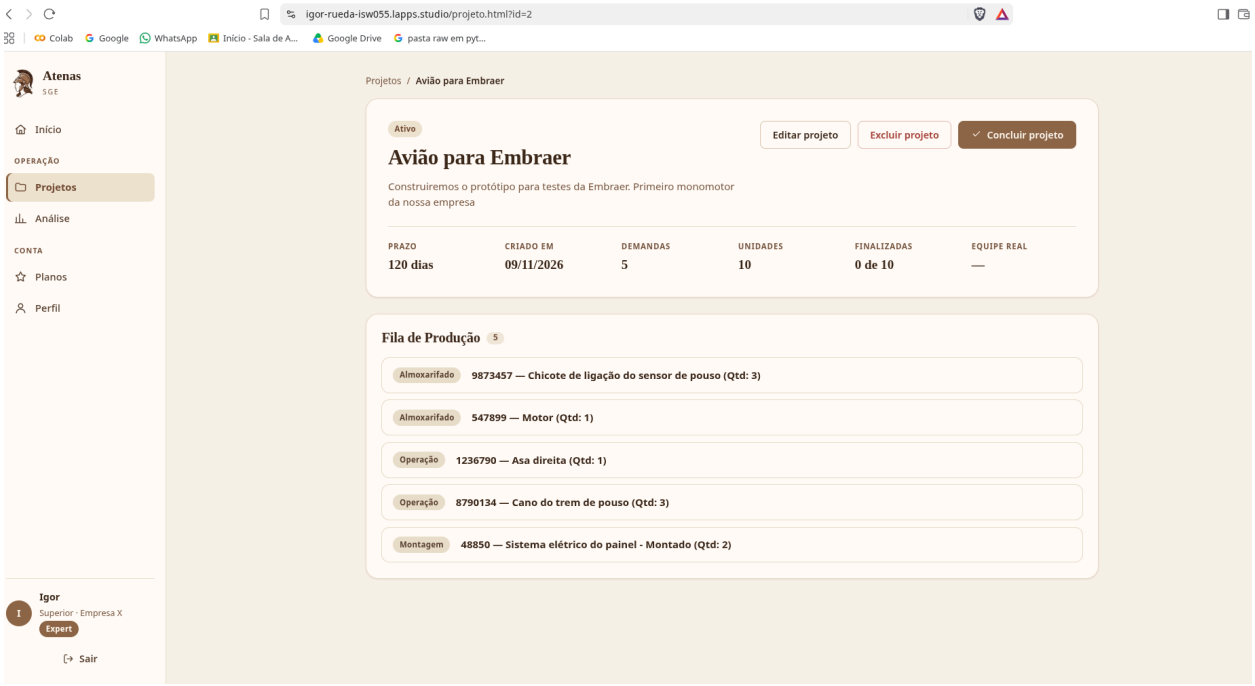


Figura 19: Atividade 10 — URL no ar (Cadastro de itens)

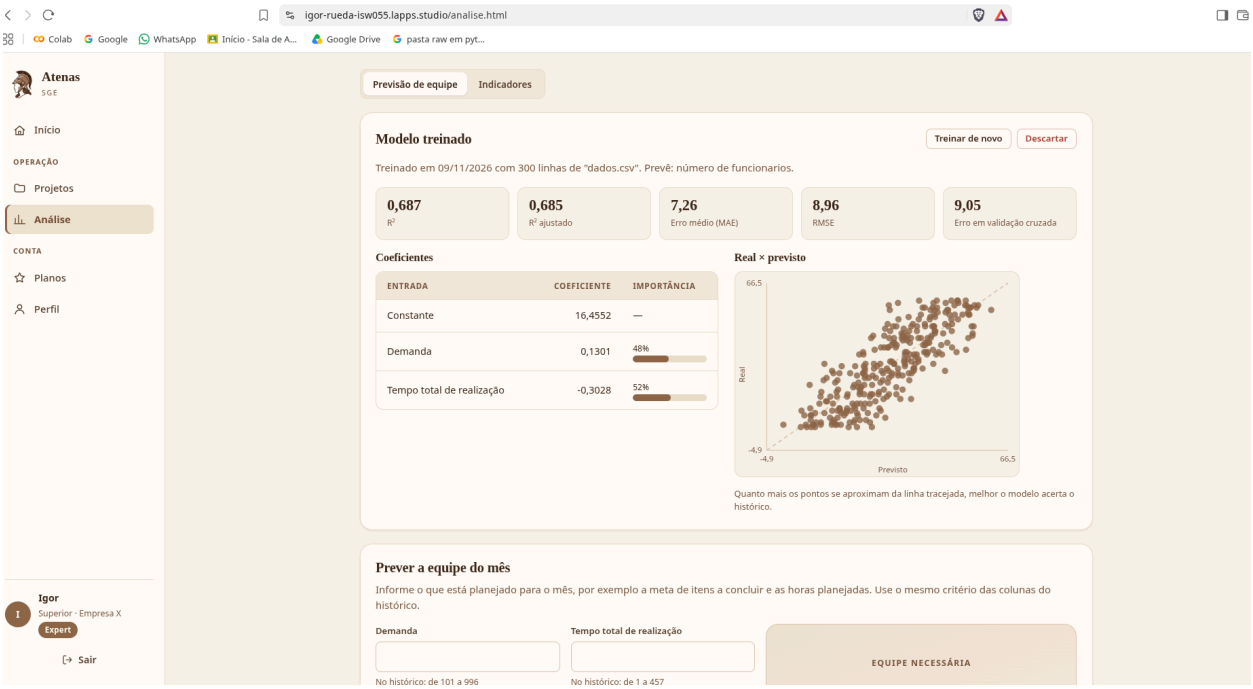


Figura 20: Atividade 10 — URL no ar (Regressão linear)

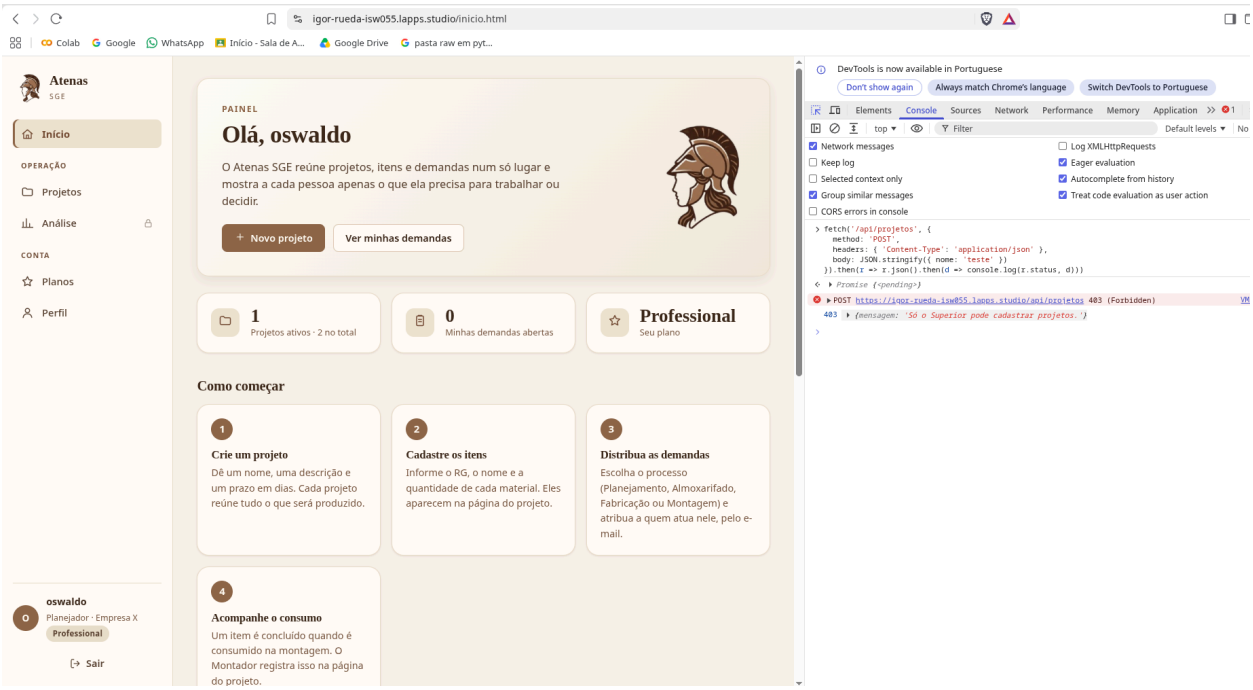


Figura 21: Atividade 10 — erro 403

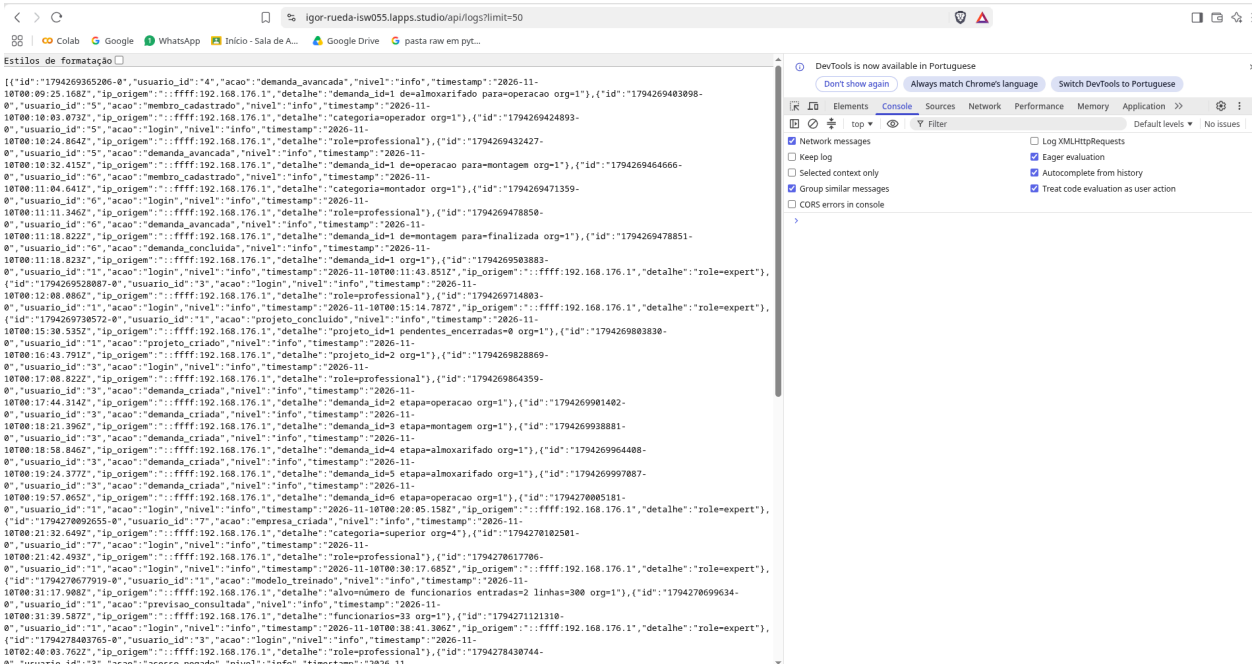


Figura 22: Atividade 10 — eventos log

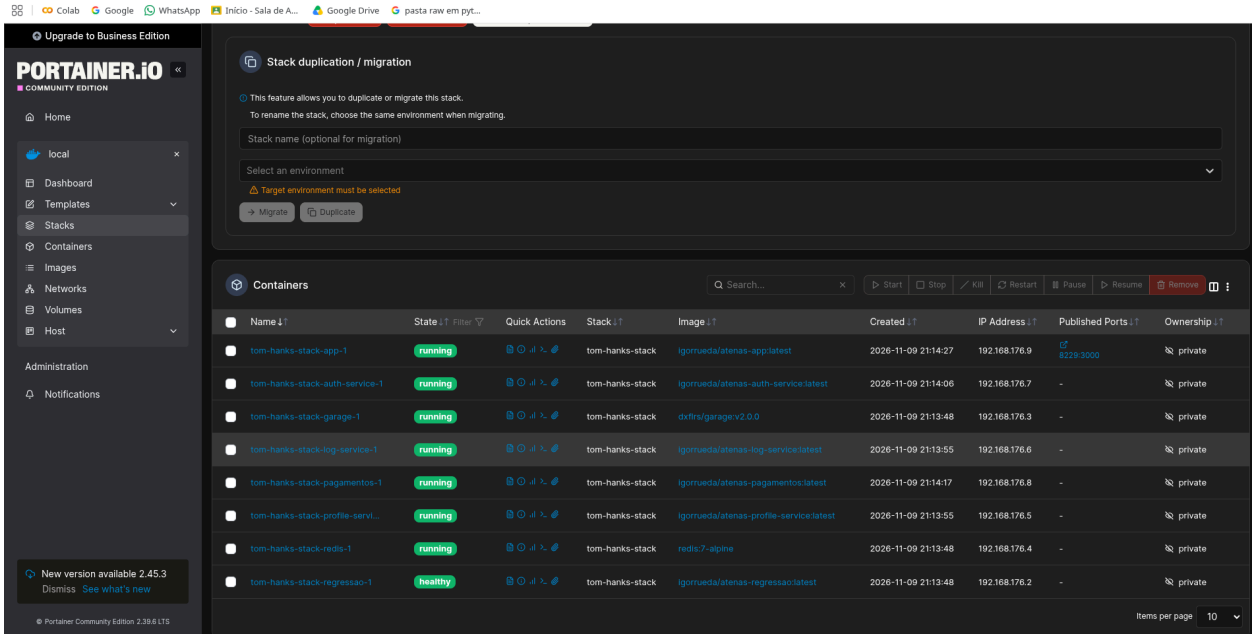


Figura 23: Atividade 10 — Saúde dos containers

Dificuldades e como foram resolvidas

A principal dificuldade ao longo desta etapa foi conciliar o alto volume de tarefas e a abrangência das alterações arquiteturais dentro do cronograma, o que acabou resultando na entrega fora do prazo. No aspecto técnico, o maior desafio concentrou-se nas migrações do banco de dados para suportar a nova estrutura multi-tenant. Isso foi resolvido estruturando a recriação do banco em scripts SQL sequenciais e isolados (base, domínio e dados iniciais), o que garantiu a integridade referencial das tabelas durante a transição do sistema.

5.5 Atividade 11 — SaaS completo — planos, limites e assinatura com Stripe

Descrição	Planos recorrentes, checkout de assinatura, webhooks atualizando o plano, portal do cliente, limites por plano na API e painel admin.
Data planejada	13/11/2026
Data realizada	12/11/2026 23:37
Situação	entregue
Evidência	GitHub — issue de entrega (@siriani) + commit + prints da assinatura com o cartão 4242, do Stripe em modo de teste, do webhook e do limite bloqueando
Link	https://github.com/Igor-Rueda/Atena-SGE

O que foi feito

Nesta atividade, foi implementado o ciclo de vida completo de assinaturas do SaaS integrado ao Stripe, englobando tanto a ativação quanto o fluxo de cancelamento de planos. O sistema passou a operar com o plano gratuito Professional, que impõe limites de uso bloqueados na API (até 3 projetos e 50 demandas), e o plano pago Expert, que remove essas travas e libera o módulo de análises. Para suportar essa operação, foram desenvolvidos o checkout recorrente, o portal do cliente — que permite o autoatendimento direto para gerenciar ou cancelar a assinatura — e um painel administrativo focado no monitoramento de assinaturas ativas, MRR e logs de transação.

A principal decisão arquitetural foi desvincular a mudança de plano da navegação da interface e atrelá-la estritamente ao processamento assíncrono dos webhooks do Stripe (utilizando eventos como checkout.session.completed para novas assinaturas e customer.subscription.deleted para o rebaixamento de contas canceladas). Para garantir a consistência dessa integração, implementou-se uma tabela de controle no banco de dados que valida a assinatura dos eventos recebidos e impede a execução duplicada de um mesmo webhook

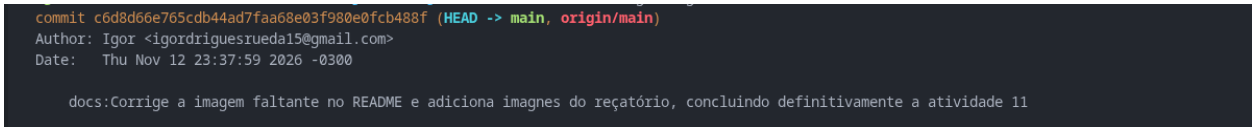


Figura 24: Atividade 11 — último commit relativo à atividade

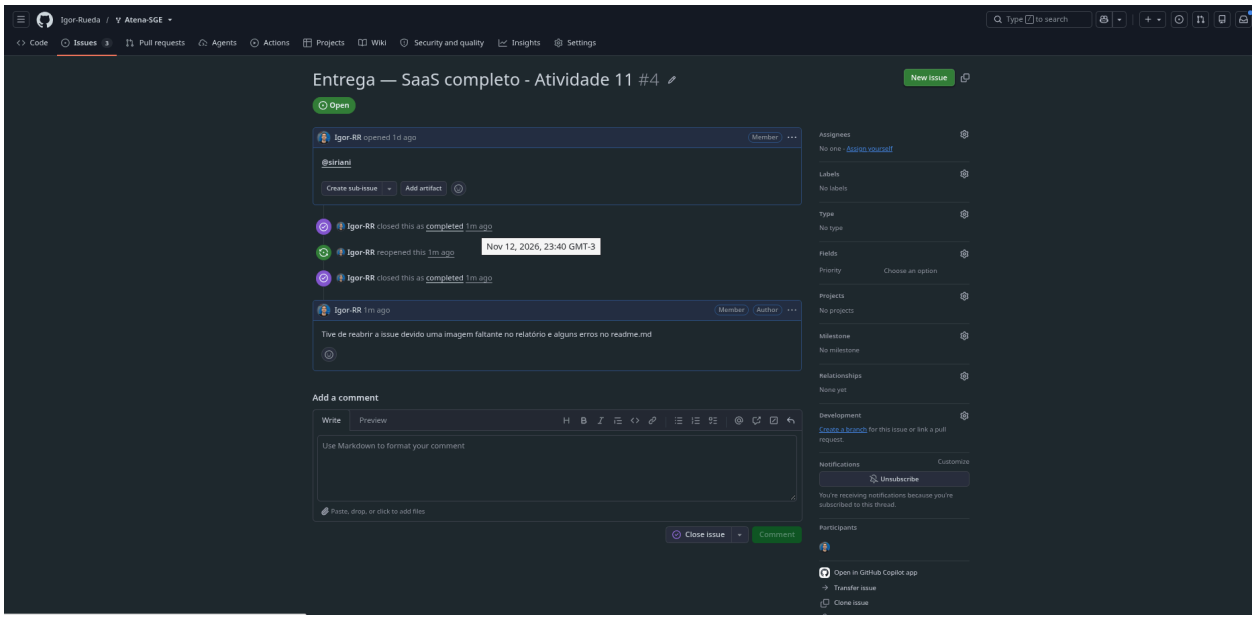


Figura 25: Atividade 11 — issue

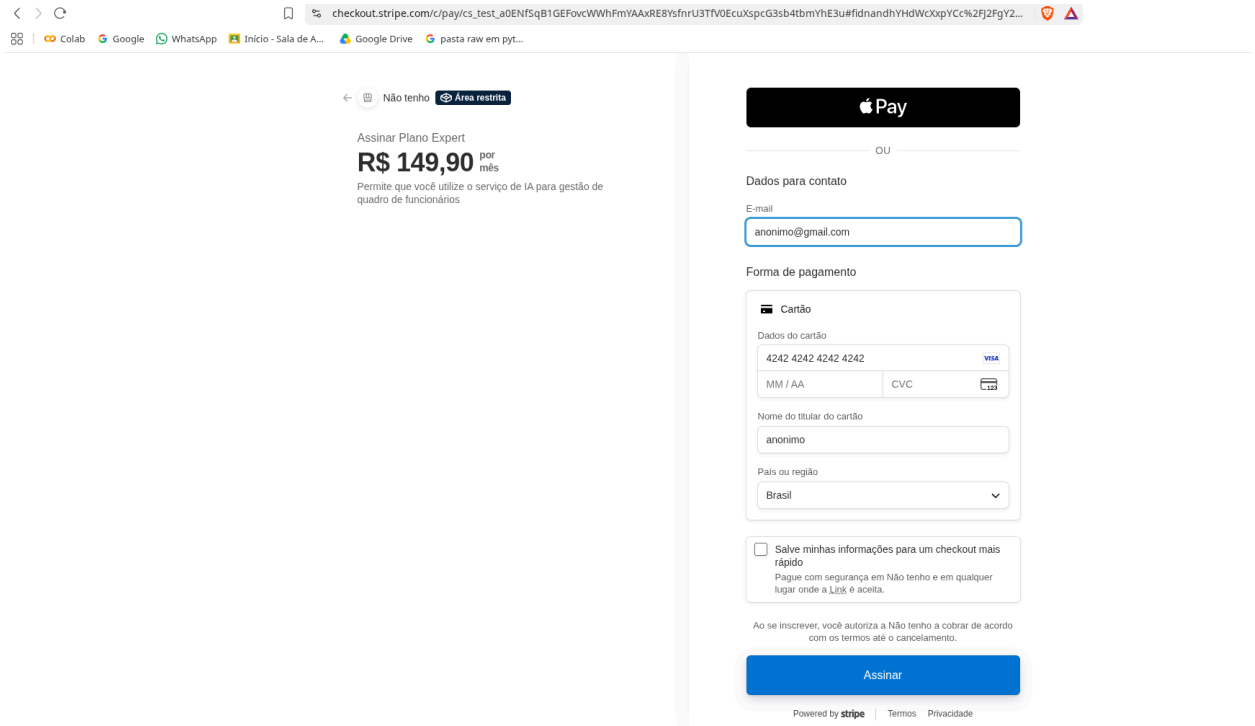


Figura 26: Atividade 11 — 4242 cartão

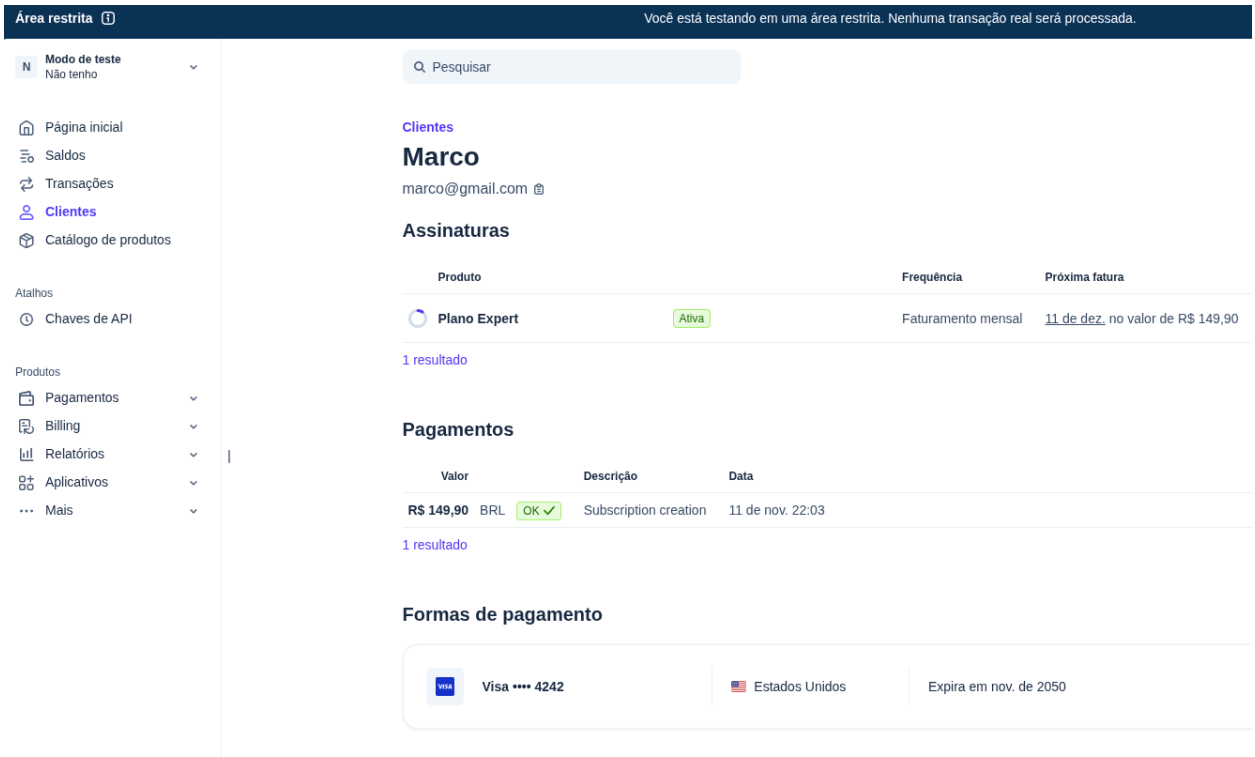


Figura 27: Atividade 11 — Stripe modo teste (o e-mail presente na imagem é um e-mail de teste)

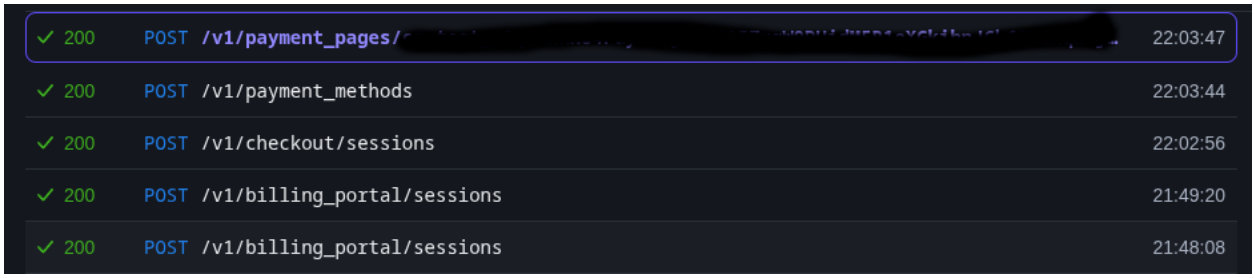


Figura 28: Atividade 11 — chekout

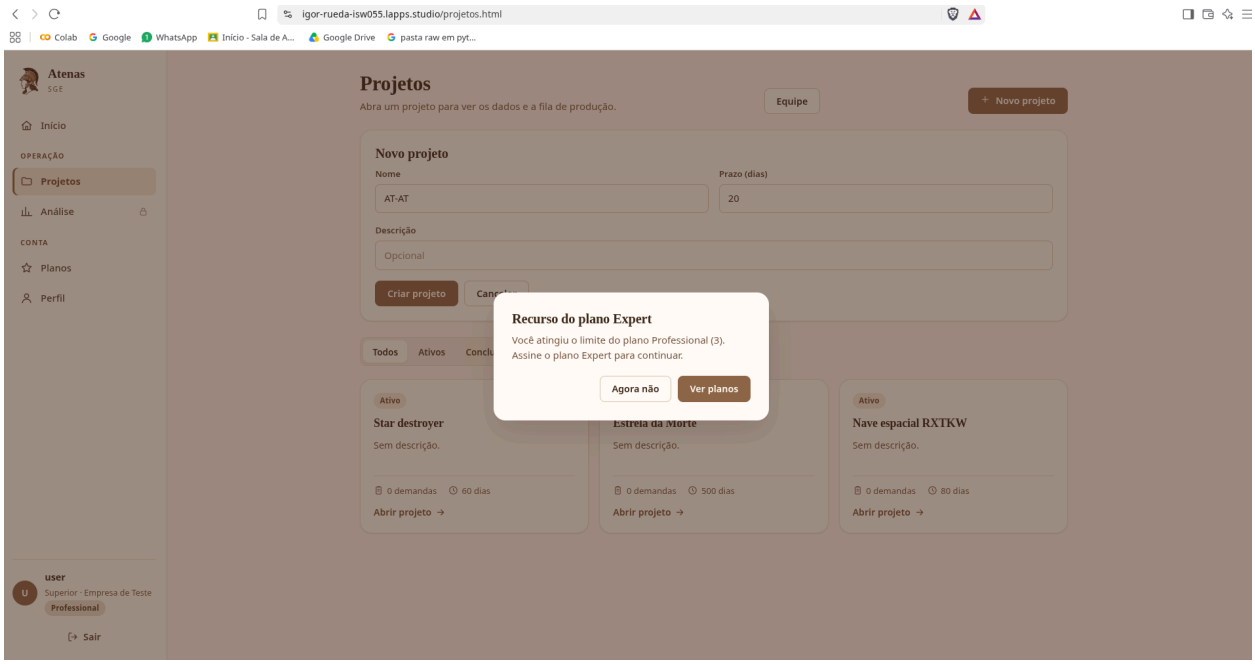


Figura 29: Atividade 11 — Limite Bloqueado

Dificuldades e como foram resolvidas.

A maior dificuldade técnica desta etapa foi adaptar o sistema para a arquitetura orientada a webhooks, o que, somado ao grande volume de tarefas da atividade, acabou resultando na entrega fora do prazo estipulado. Absorver essa lógica de pagamentos assíncronos exigiu expandir o schema do banco de dados (para espelhar o estado das assinaturas e registrar os eventos processados) e, consequentemente, realizar ajustes no pipeline de CI/CD, garantindo que o banco do ambiente de integração passasse a concatenar corretamente os novos scripts SQL para que os testes de limite e isolamento não quebrassem.

6 Considerações finais

Neste bimestre, o desenvolvimento do Atenas SGE proporcionou aprendizados que foram muito além da escrita de código, exigindo uma visão integrada entre a engenharia de software e as regras de negócio. Do lado técnico, uma das principais dificuldades que enfrentei — e um ponto de atenção para o meu aprimoramento contínuo — foi o controle de versionamento. Tive obstáculos operacionais na gerência de branches e forks, o que resultou até na exclusão acidental de uma ramificação. Esse erro prático reforçou a urgência de dominar o Git com mais segurança, especialmente em fluxos que envolvem pipelines de CI/CD. Além da infraestrutura, o projeto evidenciou que preciso refinar minha capacidade analítica voltada ao produto, melhorando a forma como identifico as necessidades reais dos usuários e como formulo e comunico as propostas de valor.

Olhando para o Atenas SGE sob a ótica de um SaaS comercial, há passos claros para que ele se torne um produto completo e maduro. Se pudesse fazer algo diferente desde o início, dedicaria mais esforço ao design da interface, garantindo uma experiência de usuário (UX) mais fluida e intuitiva para quem opera no chão de fábrica. Em relação às funcionalidades que ainda faltam, o módulo de análise (plano Expert) precisa evoluir. Embora a regressão linear múltipla atenda à prova de conceito, o próximo passo essencial é a inclusão de indicadores baseados em séries temporais e a implementação de algoritmos regressores alternativos. A adoção de modelos que se adaptem a problemas não lineares trará muito mais robustez e precisão para prever o comportamento e a alocação das equipes industriais no mundo real.

7 Declaração de autoria

Declaro que este relatório foi elaborado por mim, individualmente, e que as evidências apresentadas correspondem a entregas de minha autoria, verificáveis nos links informados.

Ígor Rodrigues Rueda

Pompeia, 13/11/2026